

SEEMULATOR — CIRCUITS DOMAIN

Circuits Platform: Bug Fixes & Architecture Report

A summary of issues found and fixed, and an in-depth walkthrough of the current request/response flow.

Prepared July 2026 — covers backend orchestrator, 10 sub-domain pipelines, and the frontend Formulated Model pane.

- 3 platform-breaking bugs fixed (startup crash, schematic renderer, frontend edit pane)
- Schematic auto-layout engine rewritten for correctness
- "Formulated Model" parameter editing/rerun extended from 1 domain to all 10
- Regression test harness added covering all 10 sub-domains

1. Errors Found & Fixed

BACKEND — STARTUP CRASH

Database initialization failure crashed the entire backend

`init_db()` re-raised any exception on DB connection failure, killing the whole FastAPI process at startup — including the Circuits domain, which doesn't even use the database. If the DB was unreachable (e.g. local dev without Postgres running), **nothing** worked.

Fix: log a warning and continue booting instead of raising, so DB-independent features (Circuits, schematic rendering, etc.) stay available even when the DB is down.

Files: `backend/db/database.py`

BACKEND — 500 ERROR

Missing streaming function caused a hard 500 on every streamed circuit request

`generate_netlist_with_repair_stream` was referenced by the API route but not defined in `netlist_ai.py`, so any request to `/api/circuits/solve/stream` failed with an HTTP 500 immediately.

Fix: implemented the missing streaming variant of the netlist-generation-with-repair loop.

Files: `backend/circuits/netlist_ai.py` · commit `a61a02f`

BACKEND — PIPELINE CRASH

`analog_sim` pipeline crashed on out-of-scope / empty netlists

When the LLM classified a question as out-of-scope (or returned an empty netlist), the pipeline had no early return and fell through into ngspice execution and schematic rendering against an empty circuit — crashing instead of returning a clean "not applicable" result.

Fix: added an explicit early return right after input generation when `in_scope` is false or the netlist is blank.

Files: `backend/domains/circuits/analog_sim/pipeline.py`

BACKEND — BROKEN SCHEMATICS

Auto-generated schematics were frequently overlapping / nonsensical

The schemdraw fallback renderer dumped every component left-to-right in a single flat loop with no awareness of circuit topology (series vs. shunt vs. source), so multi-branch circuits rendered as a jumbled, overlapping mess. It also had no handling for 3-terminal devices (BJTs) and dumped raw SPICE directive text (e.g. full `SINE(...)` / `PULSE(...)` strings) as labels.

Fix: rewrote the renderer to classify each component as *source*, *series*, or *shunt* relative to ground, walk the series chain node-by-node, drop shunt elements straight down to ground at the correct node, special-case BJTs as 3-terminal elements, and truncate/format long SPICE value strings into readable labels.

Files: `backend/domains/circuits/analog_sim/schemdraw_fallback.py`, `requirements.txt` (added `schemdraw>=0.20`)

BACKEND — SILENT SCHEMATIC FAILURES

Schematic rendering order was backwards and errors were swallowed silently

The pipeline tried the older, more fragile `lcapy`-based renderer first and only fell back to the newer, more robust `schemdraw` renderer *if `lcapy` also failed* — and a bare `except SchemdrawFallbackError: pass` meant fallback failures vanished with no log, making the actual failure impossible to diagnose.

Fix: swapped the order (schemdraw first, lcapy as fallback) and added proper `logger.warning(...)` calls at every failure point on both the sync and streaming code paths.

Files: `backend/domains/circuits/analog_sim/pipeline.py`

FRONTEND — SILENTLY DROPPED EDITS

"Apply & Run" in the Formulated Model pane did nothing for 9 of 10 sub-domains

`handleApplyChangesAndRun` only updated a parameter's value if it had a `file_anchor` (a line/substring pointer into a SPICE netlist). Only `analog_sim` parameters have a `file_anchor` — every other sub-domain's edits were matched to a parameter, then silently discarded before `updatedParameters` was ever touched. The user could edit a value, click "Apply & Run", and nothing would change.

Fix: parameters without a `file_anchor` now still get their `value` / `tag` / `oldValue` updated in the parameter list; only the netlist-line text-substitution step is skipped (it isn't needed — the backend patches the JSON plan directly by parameter `id` on rerun, see §3).

Files: `src/App.jsx` — `handleApplyChangesAndRun`

2. Feature Work: Formulated Model Sync for All 10 Sub-Domains

Before this work, only `analog_sim` (SPICE circuits) exposed an editable "Formulated Model" pane with parameters that could be tweaked and re-simulated. The other 9 sub-domains had no `model` event at all — the frontend pane was empty and rerun wasn't possible.

EXTENDED — SOLVER-BACKED DOMAINS

`symbolic_analysis`, `control_systems`, `digital_logic`, `numerical_processing`

Each pipeline now emits a `model` event with the fields that actually drive its deterministic computation, so editing them and rerunning produces a genuinely different, correctly recomputed answer:

Sub-domain	Editable field(s)	What rerun recomputes
<code>symbolic_analysis</code>	Component numeric values (R, C, L, ...)	Transfer function substitution, poles (symbolic + numeric)
<code>control_systems</code>	Numerator / denominator coefficients	Transfer function, poles, stability
<code>digital_logic</code>	Boolean expression	Full truth table, simplified expression
<code>numerical_processing</code>	Python analysis code	Re-executed against numpy/scipy, new metrics

EXTENDED — LLM-ONLY DOMAINS

`rf_em`, `pcb_realization`, `fpga_realization`, `semiconductor_device`, `physical_design`

These 5 domains have no deterministic solver behind them (the LLM directly produces the metrics), so there's nothing to "recompute." Each metric is still exposed as an editable field; on rerun the edited value is written straight into the result (no LLM re-invocation), and the chat explanation step narrates the user's edited numbers.

ORCHESTRATOR — GENERIC RERUN ENGINE

New `_apply_parameters_to_json_plan()` in `orchestrator.py`

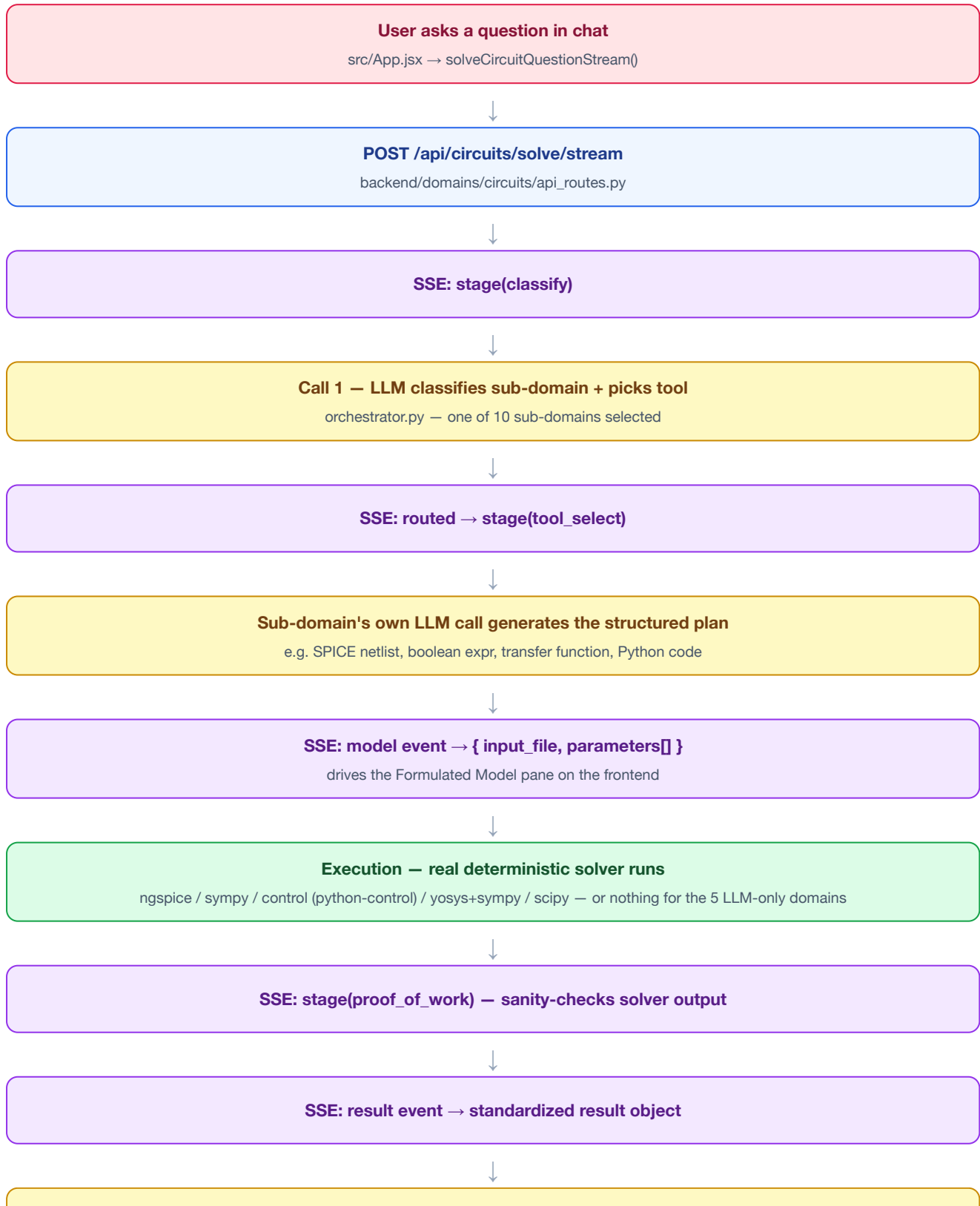
A single generic function now applies edited parameters back into any sub-domain's JSON plan using dotted-path ids (e.g. `numeric_values.R`, `metrics.0.value`, `denominator.2`) — walking nested dicts/lists, coercing types to match the original field, and skipping read-only fields so derived/nested objects are never clobbered. This single function replaces what would otherwise be 9 separate per-domain rerun implementations.

3. Current Platform Flow (In Depth)

3.1 — First-time question (normal flow)

Two LLM calls total: one to classify + generate the input, one to explain the result in chat.

■ LLM call
 ■ real solver
 ■ SSE event to frontend
 ■ frontend



Call 2 — LLM streams a natural-language explanation

token-by-token SSE events

**Chat message + Results pane + Formulated Model pane all update**

SSE: done

3.2 — Rerun (user edits a parameter and clicks "Apply & Run")

Zero or one LLM call — the sub-domain's input-generation LLM call is *skipped entirely* for 9 of 10 domains; the patched plan is reused directly.

User edits a value in the Formulated Model pane, clicks Apply & Run

src/App.jsx — handleApplyChangesAndRun()

**solveCircuitQuestionStream({ rerun: true, sub_domain, input_file, parameters })****orchestrator skips Call 1 (classify) — sub_domain is pinned by the frontend****9 domains →****_apply_parameters_to_json_plan()**

re-parses input_file as JSON, walks dotted-path ids, patches values, skips read-only fields

analog_sim →**_apply_parameters_to_input()**

rewrites the matching SPICE line using each parameter's file_anchor

**Patched plan/netlist passed as****_prebuilt_input****— sub-domain's own LLM call is skipped****SSE: model event re-emitted with fresh parameters from the patched plan****Execution re-runs the real solver against the new values (where one exists)****proof_of_work → result → Call 2 (LLM narrates the new numbers) → done**

4. All 10 Sub-Domains at a Glance

Sub-domain	Tool	Rerun input shape	Has real solver?
analog_sim	ngspice	SPICE netlist	Yes — ngspice
symbolic_analysis	sympy	JSON plan	Yes — sympy
control_systems	python-control	JSON plan	Yes — python-control
digital_logic	yosys / sympy	JSON plan	Yes — sympy boolean algebra
numerical_processing	scipy / numpy	JSON plan	Yes — executed Python code
rf_em	openems (LLM-only)	JSON plan	No — metrics are LLM-authored
pcb_realization	kicad (LLM-only)	JSON plan	No
fpga_realization	nextpnr (LLM-only)	JSON plan	No
semiconductor_device	devsim (LLM-only)	JSON plan	No
physical_design	openroad (LLM-only)	JSON plan	No

Verification performed

- Terminal test harness (backend/test_all_subdomains.py) — 6 representative questions per sub-domain run against the real orchestrator + real LLM (Cerebras provider), checking pass/fail and schematic/plot presence.
- Targeted rerun scripts for symbolic_analysis , control_systems , digital_logic , numerical_processing , and rf_em — each edited a parameter end-to-end and confirmed the rerun result reflected the new value (e.g. editing R from 1000→5000Ω moved the RC pole from −1000 rad/s to −200 rad/s exactly as expected).